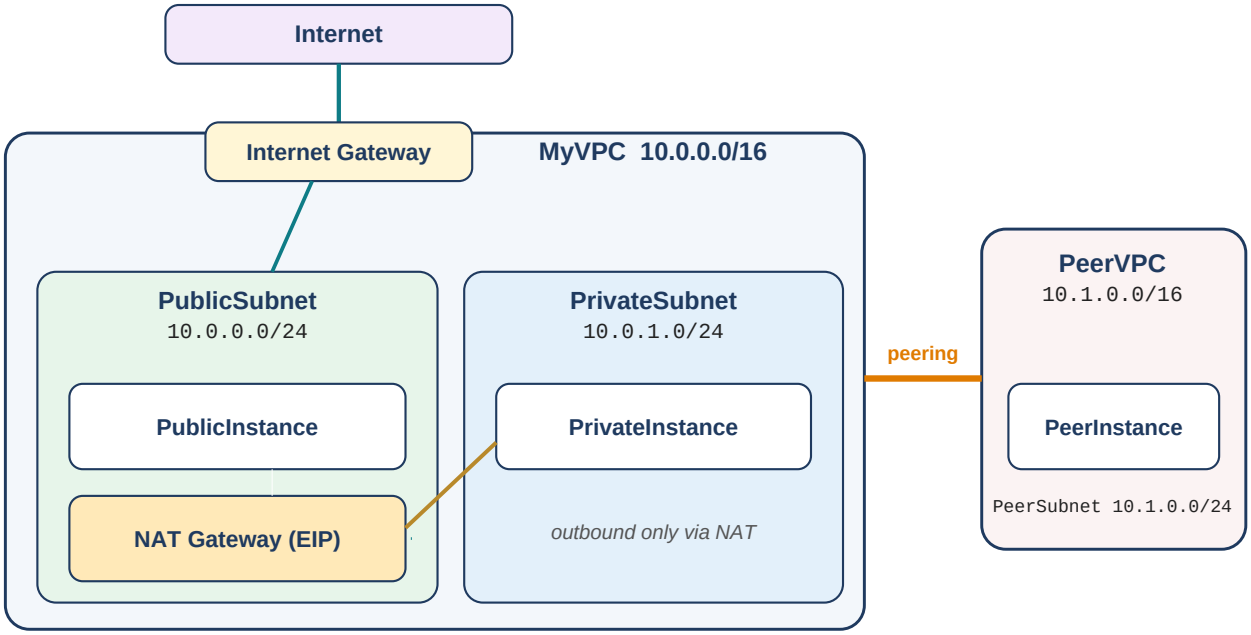
AWS VPC Lab: Step by Step

Internet Gateway, public and private instances, NAT Gateway and VPC peering. Only the steps, with checkpoints and test commands



Resource	Name	Value
VPC	MyVPC	10.0.0.0/16
Public subnet	PublicSubnet	10.0.0.0/24, AZ ap-south-1a
Private subnet	PrivateSubnet	10.0.1.0/24, AZ ap-south-1a
Internet Gateway	MyIGW	attached to MyVPC
Route tables	PublicRT / PrivateRT	public: 0.0.0.0/0 to IGW. private: 0.0.0.0/0 to NAT
NAT Gateway	MyNAT	in PublicSubnet, with an Elastic IP
Second VPC	PeerVPC	10.1.0.0/16, subnet PeerSubnet 10.1.0.0/24
Instances	PublicInstance, PrivateInstance, PeerInstance	Ubuntu 22.04, t2.micro or t3.micro

Before you start: Use one region for everything (the lab uses ap-south-1, Mumbai). **NAT Gateways and Elastic IPs cost money by the hour.** Do the Cleanup part at the end. You need a key pair (for example **MyKey.pem**) and your account must allow creating a VPC.

Part 1: Create the VPC and Two Subnets

#	Do this	Enter or choose
1	Console, search **VPC**. Check the region (top right). Open **Your VPCs**, click **Create VPC**.	Region: Asia Pacific (Mumbai) ap-south-1
2	Resources to create.	**VPC only**
3	Name tag.	**MyVPC**
4	IPv4 CIDR block.	**IPv4 CIDR manual input**, **10.0.0.0/16**. No IPv6. Tenancy: Default
5	Click **Create VPC**.	MyVPC appears in the list
6	Select MyVPC, **Actions**, **Edit VPC settings**.	Tick **Enable DNS hostnames** (DNS resolution stays on). Save
7	Left menu **Subnets**, click **Create subnet**.	VPC ID: **MyVPC**
8	Subnet 1 settings.	Name **PublicSubnet**, AZ **ap-south-1a**, IPv4 subnet CIDR **10.0.0.0/24**
9	Click **Add new subnet**. Subnet 2 settings.	Name **PrivateSubnet**, AZ **ap-south-1a**, CIDR **10.0.1.0/24**
10	Click **Create subnet**.	Both subnets are created
11	Select PublicSubnet, **Actions**, **Edit subnet settings**.	Tick **Enable auto-assign public IPv4 address**. Save

Checkpoint: Subnets list: **PublicSubnet** and **PrivateSubnet** in MyVPC, each showing **251** available IPv4 addresses (256 minus the 5 reserved by AWS).

Part 2: Internet Gateway and Public Route Table

#	Do this	Enter or choose
1	Left menu **Internet gateways**, click **Create internet gateway**.	Name tag **MyIGW**, then Create
2	Select MyIGW, **Actions**, **Attach to VPC**.	Select **MyVPC**, click **Attach internet gateway**
3	Left menu **Route tables**, click **Create route table**.	Name **PublicRT**, VPC **MyVPC**, Create
4	Select PublicRT, tab **Routes**, **Edit routes**, **Add route**.	Destination **0.0.0.0/0**, Target **Internet Gateway**, choose **MyIGW**. Save changes
5	Tab **Subnet associations**, **Edit subnet associations**.	Tick **PublicSubnet**. Save associations
6	Create a second route table.	Name **PrivateRT**, VPC **MyVPC**, Create
7	PrivateRT, tab **Subnet associations**, Edit.	Tick **PrivateSubnet**. Save. (Add no internet route)

Checkpoint: PublicRT shows two routes: **10.0.0.0/16 local** and **0.0.0.0/0 igw-xxxx**. **PrivateRT** shows only **10.0.0.0/16 local**. The IGW state is **Attached**.

Remember: A subnet is **public** because its route table has a route to the Internet Gateway. PublicSubnet is public after step 5. PrivateSubnet has no such route, so it is private.

Part 3: Launch the Public and Private Instances

A. Public instance

#	Do this	Enter or choose
1	Console, **EC2**, **Launch instance**.	Name **PublicInstance**
2	Application and OS Images.	**Ubuntu Server 22.04 LTS**
3	Instance type.	**t2.micro** (or t3.micro: Free tier eligible)
4	Key pair (login).	Select **MyKey** (or create a new one and download it)
5	Network settings, click **Edit**.	VPC **MyVPC**, Subnet **PublicSubnet**, Auto-assign public IP **Enable**
6	Firewall (security groups): create a new one.	Name **public-sg**. Rule: Type **SSH**, Source **My IP**
7	Storage: keep the default. Click **Launch instance**.	8 GiB gp3

B. Private instance

#	Do this	Enter or choose
1	**Launch instance** again.	Name **PrivateInstance**, same Ubuntu 22.04, same type, same key pair
2	Network settings, click **Edit**.	VPC **MyVPC**, Subnet **PrivateSubnet**, Auto-assign public IP **Disable**
3	Firewall: create a new security group.	Name **private-sg**
4	Rule 1.	Type **SSH**, Source type **Custom**, pick the security group **public-sg**
5	Rule 2 (for ping tests).	Type **All ICMP - IPv4**, Source **10.0.0.0/16**
6	Click **Launch instance**.	PrivateInstance has only a private IP (10.0.1.x)

Checkpoint: In Instances, wait for **Running** and **2/2 checks passed**. **PublicInstance** has a **Public IPv4 address**. **PrivateInstance** shows **-** there and a **Private IPv4** like **10.0.1.x**. Write down both.

C. Connect and test (copy and paste)

Replace **PUBLIC_IP** and **PRIVATE_IP** with the addresses you wrote down. The private instance has no public address, so you reach it **through the public instance** (a **jump host**). Do not copy your .pem file onto the server.

COPY AND PASTE: Connect to the public instance

```
cd ~/Downloads
chmod 400 MyKey.pem
ssh -i MyKey.pem ubuntu@PUBLIC_IP
```

COPY AND PASTE: Connect to the private instance through the public one

```
ssh -i MyKey.pem -J ubuntu@PUBLIC_IP ubuntu@PRIVATE_IP
```

Command	What it does (in simple words)
ssh -i MyKey.pem ubuntu@PUBLIC_IP	Normal login to the public instance.
ssh -i MyKey.pem -J ubuntu@PUBLIC_IP ubuntu@PRIVATE_IP	-J (jump) connects to PUBLIC_IP first and from there to the private instance. It works because private-sg allows SSH from public-sg.

COPY AND PASTE: Tests on the PUBLIC instance

```
ip -4 addr show
curl -s ifconfig.me; echo
ping -c 2 8.8.8.8
```

COPY AND PASTE: Tests on the PRIVATE instance (before the NAT Gateway)

```
ip -4 addr show
ip route
curl -m 5 -sI https://example.com | head -1 || echo "no internet"
ping -c 2 -W 2 8.8.8.8
```

Output (what students will see)

```
PUBLIC instance:
    inet 10.0.0.37/24 ...                  <-- private IP in PublicSubnet
13.233.10.25                            <-- its public IP
2 packets transmitted, 2 received       <-- internet works

PRIVATE instance:
    inet 10.0.1.54/24 ...                  <-- private IP only
default via 10.0.1.1 dev eth0            <-- .1 = VPC router
no internet                              <-- no route to the Internet yet
2 packets transmitted, 0 received        <-- ping fails
```

Checkpoint: Public instance can reach the Internet. Private instance **cannot**. That is correct for now. Part 4 fixes it with a NAT Gateway.

Part 4: NAT Gateway

A **NAT Gateway** lets instances in a **private** subnet reach the Internet (updates, downloads) **without** allowing anyone on the Internet to connect to them.

#	Do this	Enter or choose
1	Left menu **NAT gateways**, click **Create NAT gateway**.	Name **MyNAT**
2	Subnet.	**PublicSubnet** (the NAT Gateway must be in a **public** subnet)
3	Connectivity type.	**Public**
4	Elastic IP allocation ID.	Click **Allocate Elastic IP**
5	Click **Create NAT gateway**.	Wait until Status is **Available** (1 to 2 minutes). Refresh
6	Left menu **Route tables**, select **PrivateRT**, tab **Routes**, **Edit routes**, **Add route**.	Destination **0.0.0.0/0**, Target **NAT Gateway**, choose **MyNAT**. Save changes

Checkpoint: PrivateRT routes: **10.0.0.0/16 local** and **0.0.0.0/0 nat-xxxx**. Note the NAT's **Elastic IP** (shown on the NAT gateway page).

COPY AND PASTE: Test again on the PRIVATE instance

```
curl -s ifconfig.me; echo
ping -c 2 8.8.8.8
curl -sI https://example.com | head -1
sudo apt-get update
```

Command	What it does (in simple words)
`curl -s ifconfig.me`	Asks a website for your public IP. From the private instance it shows the **NAT Gateway's Elastic IP**.
`ping -c 2 8.8.8.8`	Replies: outbound Internet works.
`curl -sI https://example.com | head -1`	HTTPS works.
`sudo apt-get update`	Downloads updates: the main reason private servers need NAT.

Output (what students will see)

```
13.126.40.12                              <-- the NAT Gateway's Elastic IP
2 packets transmitted, 2 received         <-- ping works now
HTTP/2 200                                <-- HTTPS works
Get:1 http://ap-south-1.ec2.archive.ubuntu.com ...   <-- updates download
```

Checkpoint: The private instance has **no public IP** and cannot be reached from the Internet, yet it can go **out**. The address websites see is the NAT's Elastic IP.

Part 5: VPC Peering

VPC peering connects two VPCs so they can talk using **private IP addresses**. The two VPCs must have **different, non-overlapping CIDR ranges**, so we use 10.0.0.0/16 and 10.1.0.0/16.

A. Create the second VPC and an instance

#	Do this	Enter or choose
1	VPC, **Your VPCs**, **Create VPC**.	VPC only, Name **PeerVPC**, CIDR **10.1.0.0/16**
2	**Subnets**, **Create subnet**.	VPC **PeerVPC**, Name **PeerSubnet**, AZ ap-south-1a, CIDR **10.1.0.0/24**
3	EC2, **Launch instance**.	Name **PeerInstance**, Ubuntu 22.04, same key pair
4	Network settings, **Edit**.	VPC **PeerVPC**, Subnet **PeerSubnet**, Auto-assign public IP **Disable**
5	Security group: create new.	Name **peer-sg**. Rule 1: **SSH**, Source **10.0.0.0/16**. Rule 2: **All ICMP - IPv4**, Source **10.0.0.0/16**
6	Click **Launch instance**.	Note its **Private IPv4**, for example 10.1.0.25

B. Create and accept the peering connection

#	Do this	Enter or choose
1	VPC dashboard, **Peering connections**, **Create peering connection**.	Name **MyVPC-PeerVPC**
2	Requester.	VPC ID **MyVPC**
3	Accepter.	Account **My account**, Region **This Region**, VPC ID **PeerVPC**
4	Click **Create peering connection**.	Status: **Pending acceptance**
5	Select it, **Actions**, **Accept request**, confirm **Accept request**.	Status becomes **Active**

Note: If the other VPC is in a **different account**, log in to that account, open Peering connections and accept the request there.

C. Add routes in BOTH VPCs

#	Do this	Enter or choose
1	Route tables, select **PublicRT** (MyVPC), Routes, **Edit routes**, Add route.	Destination **10.1.0.0/16**, Target **Peering Connection**, choose **MyVPC-PeerVPC** (pcx-xxxx). Save
2	(Optional) Same route in **PrivateRT** if the private instance must reach PeerVPC.	Destination 10.1.0.0/16, Target the same peering connection
3	Route tables, select the route table of **PeerVPC** (its main route table), Routes, Edit routes, Add route.	Destination **10.0.0.0/16**, Target **Peering Connection** (pcx-xxxx). Save

D. Security groups and test

peer-sg already allows SSH and ICMP from **10.0.0.0/16**. Outbound rules allow everything by default. Now test from the **public instance** (replace PEER_PRIVATE_IP, for example 10.1.0.25):

COPY AND PASTE: Test peering from the PublicInstance

```
ping -c 3 PEER_PRIVATE_IP
```

COPY AND PASTE: Log in to PeerInstance from your own computer through the public instance

```
ssh -i MyKey.pem -J ubuntu@PUBLIC_IP ubuntu@PEER_PRIVATE_IP
ip -4 addr show
```

Output (example)

```
3 packets transmitted, 3 received, 0% packet loss    <-- peering works
    inet 10.1.0.25/24 ...                                  <-- inside PeerVPC
```

Peering rule	Meaning
No overlapping CIDRs	The two VPCs must use different ranges.
Routes on both sides	Each VPC needs a route to the other VPC's CIDR.
Security groups	They must allow the traffic (here: from 10.0.0.0/16).
Not transitive	If A peers with B and B peers with C, A cannot reach C through B.
Works across	Different accounts and different regions are possible.

If Something Does Not Work

Problem	Check
Cannot SSH to the public instance	Public IP assigned? PublicRT has 0.0.0.0/0 to MyIGW and is associated with PublicSubnet? public-sg allows SSH from your current IP?
Cannot SSH to the private instance	private-sg allows SSH from public-sg? Using -J through the public instance? Same key pair?
Private instance has no Internet after NAT	NAT Gateway state Available and in the **public** subnet? PrivateRT route 0.0.0.0/0 to the NAT? PrivateSubnet associated with PrivateRT?
Ping to the peer fails	Peering status Active? Routes added in both VPCs? peer-sg allows ICMP from 10.0.0.0/16? CIDRs do not overlap?

Cleanup (in this order)

Cost: Do this when the lab is finished. The **NAT Gateway** (hourly plus data) and an unused **Elastic IP** keep costing money if you forget them.

#	Do this	Enter or choose
1	EC2, Instances: select the three instances, **Instance state**, **Terminate**.	PublicInstance, PrivateInstance, PeerInstance. Wait for Terminated
2	VPC, Peering connections: select, **Actions**, **Delete peering connection**.	Also tick the option to delete related routes
3	VPC, NAT gateways: select MyNAT, **Actions**, **Delete NAT gateway**.	Wait until Deleted
4	EC2, Elastic IPs: select the NAT's address, **Actions**, **Release Elastic IP addresses**.	Release it
5	VPC, Internet gateways: MyIGW, **Actions**, **Detach from VPC**, then **Delete internet gateway**.	
6	VPC, Subnets: delete PublicSubnet, PrivateSubnet and PeerSubnet.	
7	VPC, Route tables: delete PublicRT and PrivateRT (remove associations first).	The main route table goes with its VPC
8	VPC, Your VPCs: delete MyVPC and PeerVPC.	Custom security groups can be deleted in EC2, Security Groups

Final checklist	Done?
Public instance reachable by SSH and has Internet	[ ]
Private instance has no public IP but reaches the Internet through NAT	[ ]
curl ifconfig.me on the private instance shows the NAT's Elastic IP	[ ]

Final checklist	Done?
Ping works between the public instance and PeerInstance	[ ]
Everything deleted and the Elastic IP released	[ ]